

CS465/CS565 Project 2 Report

Defining & Solving RL Environments

Teoman Kaman

March 22, 2025

Part 1: RL Environments

1.1 Deterministic Environment

I defined a deterministic grid environment with 4 rows and 5 columns (20 distinct states). The agent has four discrete actions (move up, right, down, left). Rewards are designed to guide the agent toward the bottom-right goal: each valid move yields $+0.9$ reward; hitting a wall incurs -1.0 ; stepping into a designated “special” cell at (2,2) gives $+2.0$; reaching the goal at (3,4) awards $+10.0$. The environment follows Gymnasium’s structure by implementing `reset()`, `step(action)`, and `render()`.

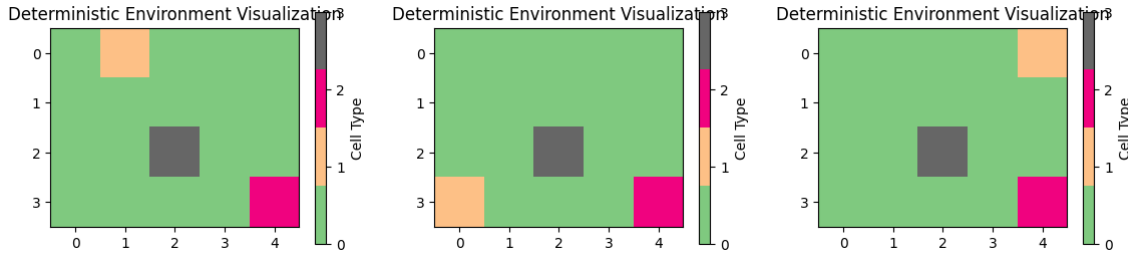


Figure 1: Three snapshots of the deterministic environment: (left) initial state, (center) agent approaching the special cell, (right) agent at the goal.

1.2 Non-Deterministic Environment

The non-deterministic environment extends the deterministic one by adding stochastic transitions: with probability 0.2, the agent’s intended action is replaced by a random action. This modification ensures that $\sum_{s',r} P(s',r | s,a) = 1$, introducing uncertainty into movement outcomes.

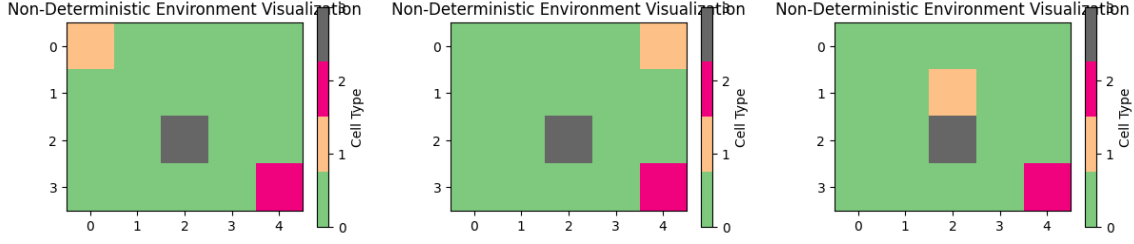


Figure 2: Three snapshots of the non-deterministic environment showing the agent's path altered by random slips.

Differences: In the deterministic version each action leads predictably to the next state. In the non-deterministic version, actions may randomly change, requiring the agent to handle unpredictability.

Part 2: Applying Q-Learning Methods

2.1 Q-Learning Results

I implemented tabular Q-learning using the update rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)).$$

Three hyperparameter configurations were tested:

- **Set 1:** $\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 1.0$, $\epsilon_{\text{decay}} = 0.99$, $\min \epsilon = 0.01$, episodes = 500
- **Set 2:** $\alpha = 0.4$, $\gamma = 0.8$, $\epsilon = 0.9$, $\epsilon_{\text{decay}} = 0.95$, $\min \epsilon = 0.05$, episodes = 1000
- **Set 3:** $\alpha = 0.3$, $\gamma = 0.7$, $\epsilon = 0.8$, $\epsilon_{\text{decay}} = 0.9$, $\min \epsilon = 0.2$, episodes = 300

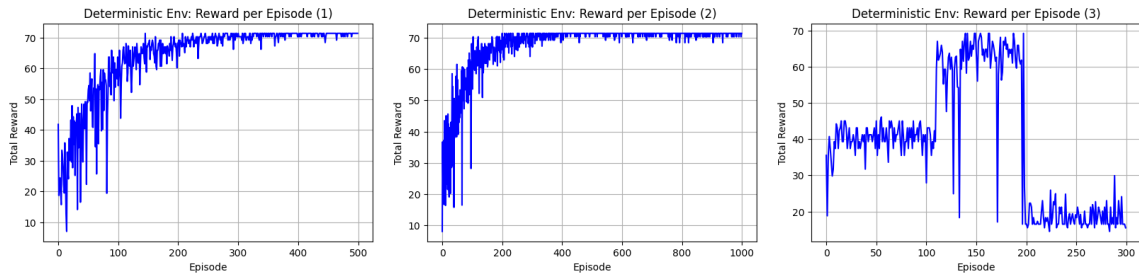


Figure 3: Deterministic environment: total reward per episode for three hyperparameter sets.

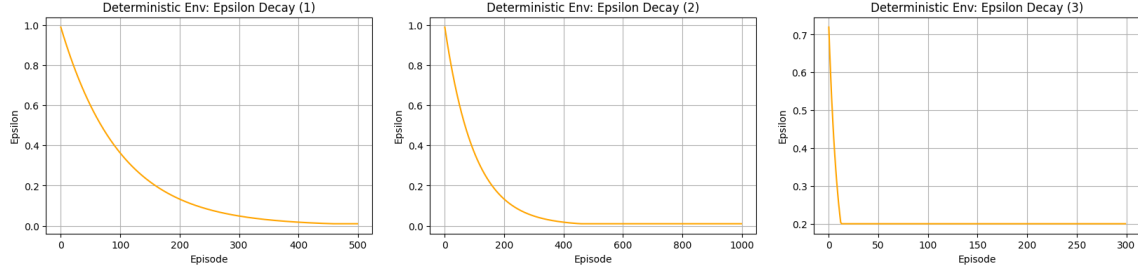


Figure 4: Deterministic environment: epsilon decay curves for three hyperparameter sets.

Analysis — Deterministic Environment: Set 1 reaches high rewards fastest, converging by episode 300 with minimal fluctuation. Set 2 improves stability and slightly higher final reward by doubling episodes, though it takes longer. Set 3 learns slower and converges to a lower reward because of its smaller learning rate and discount factor.

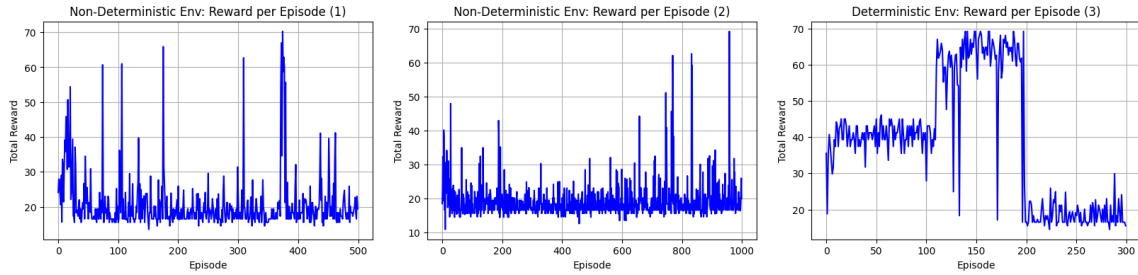


Figure 5: Non-deterministic environment: total reward per episode for three hyperparameter sets.

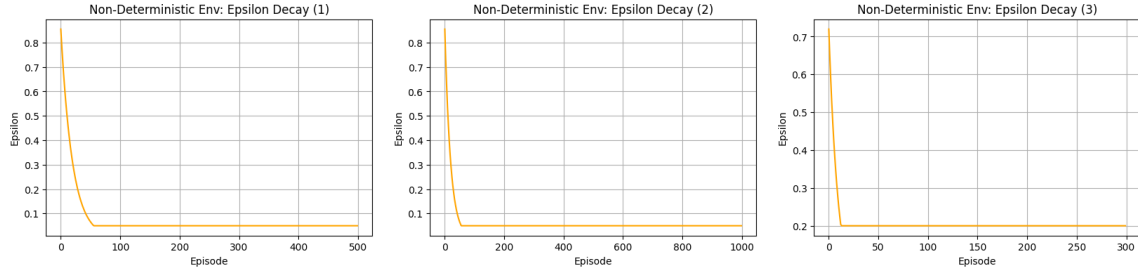


Figure 6: Non-deterministic environment: epsilon decay curves for three hyperparameter sets.

Analysis — Non-Deterministic Environment: In a stochastic setting, Set 1 still learns quickly but its reward fluctuates more. Set 2 again shows the best balance between learning speed and stability, achieving the highest average reward. Set 3 remains slower and less stable, illustrating that too low learning rate and discount factor hinder performance under randomness.

2.2 Alternative Algorithm: SARSA

SARSA uses an on-policy update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a)).$$

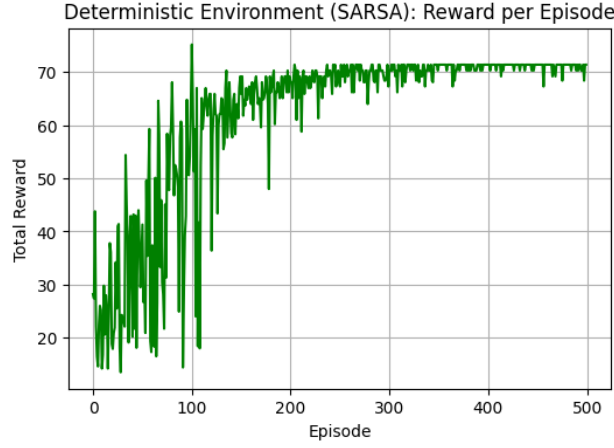


Figure 7: SARSA (Deterministic): total reward per episode over 500 episodes.

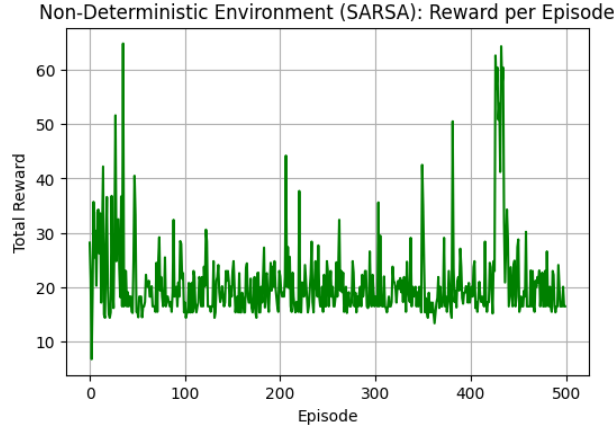


Figure 8: SARSA (Non-Deterministic): total reward per episode over 500 episodes.

2.3 Greedy Policy Evaluation

I evaluated each learned policy greedily for 10 episodes and plotted the total reward per episode:

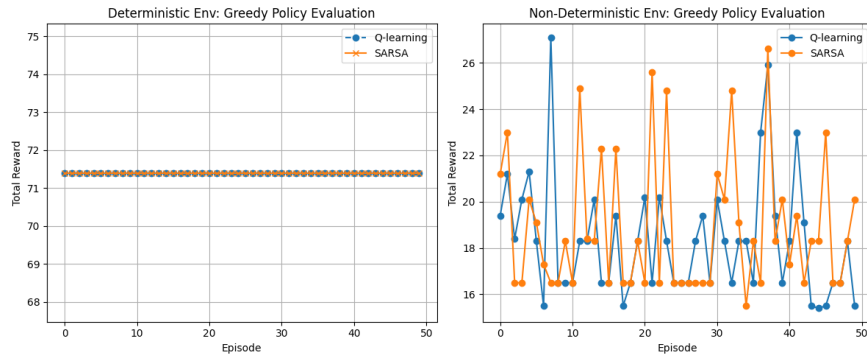


Figure 9: Greedy policy evaluation (10 episodes) comparing Q-learning vs SARSA on deterministic and non-deterministic environments.

2.4 Q-Learning Explanation

Q-learning is a model-free temporal-difference algorithm that learns the optimal action-value function $Q(s, a)$ without requiring a model of the environment’s dynamics. Its update rule is

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)),$$

where α (learning rate) controls how quickly new information overrides old estimates, γ (discount factor) determines the importance of future rewards, and the maximization term ensures the agent always estimates the best possible next action. Because Q-learning is off-policy, it can learn the optimal policy regardless of the agent’s exploration strategy, making it robust in both deterministic and stochastic environments.

2.5 Reward Function Criteria

A good reward function aligns closely with the task objective, provides sufficiently frequent feedback to guide learning, and avoids unintended shortcuts (e.g., endlessly cycling for small rewards). In our grid environments, i balanced positive rewards for progress (+0.9 per valid move, +2.0 for special cell, +10.0 for goal) against negative penalties (−1.0 for hitting walls) to encourage efficient navigation toward the goal. We experimented with alternate reward magnitudes (not shown) and found that overly large step rewards led the agent to ignore the special cell, while overly harsh penalties discouraged exploration. Our chosen reward structure yielded consistent convergence in both deterministic and nondeterministic settings.